

AGENCYOS

QA, Testing & Production Readiness System

Document 14 of the AgencyOS Master Documentation Set

360° TESTING • BUG SEVERITY • REGRESSION • SECURITY • PERFORMANCE •
PRODUCTION-READINESS SCORE • DEPLOYMENT GATES

Purpose: Define an independent, evidence-driven quality system that verifies the complete project before deployment, measures defects and risk, produces a production-readiness score, and prevents incomplete or unsafe releases from being deployed.

1. Core Principle

- QA is an independent verification layer. A developer agent's claim that code is complete is never sufficient evidence that the product is production-ready.
- QA validates against approved scope, UI, acceptance criteria and technical requirements.
- Testing covers functional and non-functional behavior.
- Critical defects block release.
- Regression testing protects previously working functionality.
- Security and performance are explicit release dimensions.
- Production readiness is evidence-based.
- Deployment gates are deterministic and policy-controlled.
- Admin can override supported gates only with an explicit, audited exception.
- A release candidate must map back to an exact source/build version.

2. End-to-End QA Lifecycle

- DEVELOPMENT COMPLETE → QA INTAKE → TEST PLAN → BUILD VALIDATION → FUNCTIONAL TESTING → INTEGRATION → E2E → REGRESSION → SECURITY → PERFORMANCE → COMPATIBILITY → DEFECT FIX LOOP → FINAL VERIFICATION → READINESS SCORE → DEPLOYMENT GATE.
- Testing scope is configurable by project type.
- Critical checks cannot be silently skipped.
- Failed gates remain visible until resolved or formally overridden.

3. QA Inputs

- Active scope version; approved UI version; approved prototype where applicable; acceptance criteria; feature/requirement mappings; release candidate/build ID; environment configuration; API/data contracts; supported platforms/devices; security requirements; performance targets; known limitations; previous defect history; change-request history.
- QA tests the approved baseline, not an agent's interpretation of what the project was supposed to be.

4. QA Agent Workforce

- QA Orchestrator/Lead Agent.
- Functional Test Agent.
- UI/E2E Test Agent.
- API/Integration Test Agent.
- Database Test Agent.
- Security Test Agent.
- Performance Test Agent.
- Compatibility/Device Test Agent.
- Regression Test Agent.
- Release/Production Readiness Agent.
- The Orchestrator assigns specialists and consolidates evidence.

5. QA Test Plan

- Test scope; required categories; supported environments; critical journeys; acceptance criteria; risk areas; mandatory release gates; performance targets; security checks; regression suite; test-data strategy; defect severity rules;

readiness-score configuration.

6. 360° Testing Model

- 360° QA = FUNCTIONAL + UI + API + DATABASE + INTEGRATION + E2E + REGRESSION + SECURITY + PERFORMANCE + COMPATIBILITY + DEPLOYMENT/SMOKE.
- Not every category has identical depth on every project; mandatory categories derive from project type, technology, scope and risk.
- Critical paths receive higher testing priority.

7. Functional Testing

- Feature behavior, business rules, validation, CRUD operations, permissions, notifications, payments where applicable, search/filter/sort, forms, error handling, success states, boundary conditions and acceptance criteria.
- Each requirement/feature should have test coverage or an explicit reason why no direct test is applicable.

8. UI Testing

- Screen availability, navigation, responsive layouts, typography/layout consistency, controls, loading/empty/error/success states, form validation, accessibility, visual regression where configured, and approved UI-to-build consistency.
- QA compares implementation against the exact approved UI version.

9. API & Database Testing

- API: endpoint availability, request validation, response schema, authentication, authorization, errors, rate limits, pagination/filtering, data integrity, idempotency and backward compatibility.
- Database: schema, CRUD, constraints, relationships, migrations, rollback strategy where applicable, data validation, authorization boundaries, transactions, critical-query performance and destructive-change checks.

10. Integration & End-to-End Testing

- Test frontend↔backend, backend↔database, third-party services, authentication, payments where applicable, WhatsApp/communication integrations, AI providers, storage and deployment infrastructure.
- E2E tests represent real user outcomes: authentication, core business transaction, primary project workflow, payment/milestone workflow where included, Admin workflow, client review/approval and production smoke journey.

11. Regression Testing

- Every significant change must be checked for unintended impact on accepted behavior.
- Run targeted regression after changes and full relevant regression before release.
- High-risk changes receive expanded regression.
- Add regression tests for important escaped defects.
- Regression failures remain release evidence, not optional noise.

12. Smoke Testing

- Application starts; critical routes load; authentication works where applicable; critical APIs respond; database connectivity works; primary workflow can begin; assets load; no immediate fatal errors.
- Smoke testing is the minimum post-deployment confidence check and does not replace full QA.

13. Defect Management

- Every bug stores: Bug ID, project/build, environment, feature/requirement, scope/UI version, title, description, reproduction steps, expected/actual result, evidence, severity, priority, frequency, detector, assignee, status, fix commit/build, retest and regression result.
- Lifecycle: OPEN → TRIAGED → ASSIGNED → IN_PROGRESS → FIX_READY → QA_RETEST → VERIFIED → CLOSED.
- Possible states: REOPENED, BLOCKED, DUPLICATE, WON'T_FIX, DEFERRED.
- Defects are not closed by status change alone; QA verification is recorded.

14. Bug Severity

- S0 / BLOCKER — release impossible, critical outage, catastrophic security/data impact → immediate block.
- S1 / CRITICAL — core function broken or severe business/security/data impact → block release.
- S2 / HIGH — important feature materially broken → normally block or explicit exception.
- S3 / MEDIUM — non-critical functional/UX issue → may release if policy allows.
- S4 / LOW — minor cosmetic/documentation issue → normally non-blocking.
- Severity describes impact; priority describes urgency. Exact thresholds are Admin-configurable.

15. Security Testing

- Authentication, authorization/role isolation, session handling, input validation, injection risks, XSS/CSRF where applicable, secret exposure, file uploads, API access control, rate limiting, dependency vulnerabilities, sensitive-data handling, privacy/logging and configuration security.
- Critical security findings block release unless an authorized policy exception explicitly permits release.
- No hard-coded production secrets, exposed API keys or unrestricted administrative endpoints.

16. Performance Testing

- Page/screen load time, API latency, database-query performance, concurrency, throughput, resource usage, memory/CPU behavior, network payload size, startup time and critical workflow completion time.
- Expected-load, peak-load, stress and endurance testing are configurable by project risk.
- Targets must be project-specific; AI must not invent universal thresholds.

17. Compatibility & Test Data

- Declare supported browsers, OS versions, mobile devices, screen sizes, network conditions, input methods, accessibility configurations and third-party versions.
- Use synthetic/test data where possible; separate test and production data; maintain resettable fixtures; include edge cases and masked sensitive data where required.

18. Automated & Manual Testing

- Automation can include CI tests, unit, integration, E2E, static analysis, security scans, build verification, regression and performance smoke checks.
- Manual/exploratory testing covers usability, unusual input combinations, visual inspection, edge workflows, client-specific acceptance and areas with weak automation.
- Automated results should be machine-readable and linked to the build.

19. Production Readiness Score

- Readiness summarizes evidence across functional coverage, regression health, security, performance, compatibility, critical-path E2E, defect health, build/release health, monitoring/observability and documentation/handover.

- Each dimension receives a configured score based on objective evidence.
- Weights vary by project type/risk.
- Critical hard gates can override the aggregate score.
- A 95/100 score cannot authorize release if a critical security gate fails.
- The scoring model and weights are configurable in the Admin Policy Engine.

20. Suggested Readiness Bands

Band	Interpretation	Action
90–100	Strong production readiness	Eligible if all hard gates pass
80–89	Generally ready with controlled issues	Review exceptions
70–79	Risk remains	Normally hold/rework
<70	Not ready	Block release

21. Hard Deployment Gates

- Build succeeds.
- Critical tests pass.
- No unresolved S0/S1 defects.
- Required security gates pass.
- Required performance gates pass.
- Required database-migration validation passes.
- Release artifact is uniquely identified.
- Deployment configuration is valid.
- Rollback plan exists where required.
- Required Admin/PM approval exists.
- Required client acceptance exists where contractual workflow requires it.

22. Release Candidate

- Candidate ID, source commit, build ID, version, environment, test report, security report, performance report, open defects, readiness score, approval status and rollback artifact/strategy.
- After approval, the candidate should be immutable or uniquely identifiable; a different commit requires a new candidate.

23. Pre-Production & Post-Deployment Gates

- FINAL QA → RELEASE CANDIDATE → READINESS SCORE → HARD GATES → APPROVAL → PRODUCTION.
- After deployment: application start, critical route, authentication, database, critical API, primary journey, monitoring/logging and runtime-error smoke checks.
- Failure triggers rollback or escalation according to policy.

24. Rollback & Observability

- Identify previous stable version; document/automate rollback; assess database/configuration rollback; use feature flags where useful; define authorization; run smoke tests after rollback.
- Monitor application health, errors, latency, resources, critical business events, external integrations, database health, security alerts and logs/traces/metrics where applicable.

25. Release Exception System

- Exception ID, failed gate, risk assessment, business reason, mitigation, expiry/review date, approver, build/version, client impact and rollback plan.
- Exceptions must be explicit. Agents must never silently ignore failed gates.

26. Admin Controls

- Configure test requirements, severity rules, readiness weights, hard gates, release exceptions, production deployment, rollback, performance/security thresholds, and emergency deployment stop.
- View test reports, defect trends, build history and agent evidence.
- Override supported gates only with reason and audit trail.

27. QA Orchestrator Responsibilities

- Create test plan, select agents, build test matrix, coordinate execution, aggregate evidence, triage defects, trigger regression, calculate readiness, enforce hard gates, prepare release recommendation and escalate unresolved risk.
- QA recommends readiness; Policy determines whether deployment is allowed.

28. Developer ↔ QA Handoff

- Developer supplies exact commit/build, implementation summary, tests/results, known limitations and verification instructions.
- QA reproduces from a defined environment and independently validates acceptance criteria.
- Defects link to exact build; fixes return through the same evidence loop.

29. Client Acceptance

- Client reviews the approved deliverable where contractually required.
- Approval references exact version/build.
- Feedback is classified; new requests become change requests.
- Final acceptance is retained for handover/completion.
- Client acceptance does not replace technical QA.

30. QA Memory & Dashboard

- Retain test plans, test cases, build history, defect history, regression history, security findings, performance results, readiness scores, release exceptions, deployment/rollback events and client acceptance.
- Dashboard shows release candidate, readiness score, hard-gate status, tests passed/failed/skipped, critical-path coverage, defects by severity, regression, security, performance, compatibility, build, deployment, rollback readiness and approvals.

31. Preventing False Production Readiness

- Never use agent confidence as evidence.
- Skipped tests are not passes.
- Critical defects cannot be hidden inside an average score.
- Do not approve a build different from the tested build.
- Do not ignore regression failures.
- Compilation is not functional correctness.
- Prototype success is not production readiness.

- Deployment requires required rollback and monitoring readiness.
- A high score cannot override a hard safety or quality gate.

32. Example Release Flow

- Developer marks build ready → QA validates → tests execute → defects triaged → fixes verified → regression passes → security passes → performance passes → readiness score calculated → hard gates pass → deployment approval → production → smoke tests → monitoring → release successful.
- If a critical security finding appears at any point, release becomes BLOCKED regardless of numerical score.

33. Acceptance Criteria

- QA is independent from development completion.
- Test plans and 360° categories are configurable.
- Functional, UI, API, database, integration and E2E testing are supported.
- Regression is required for configured changes.
- Security, performance and compatibility are explicit.
- Defects have severity and priority.
- Critical defects block release.
- Readiness score is evidence-based and configurable.
- Hard gates override aggregate score.
- Release candidate maps to exact commit/build.
- Post-deployment smoke testing is supported.
- Rollback and observability readiness are tracked.
- Exceptions are explicit and audited.
- Admin can control supported policies.
- Agents cannot declare production-ready without required evidence.

34. Final Architecture

DEVELOPMENT BUILD → QA INTAKE → 360° TESTING → BUG TRIAGE → FIX/RETEST → REGRESSION → SECURITY → PERFORMANCE → COMPATIBILITY → FINAL EVIDENCE → READINESS SCORE → HARD GATES → APPROVAL → DEPLOY → SMOKE TEST → MONITOR → RELEASE

The QA, Testing & Production Readiness System is the final quality-control layer before AgencyOS allows a project to move toward production and handover.

Key outcome: AgencyOS should be able to answer, with evidence, what exactly is being deployed, what was tested, what defects remain, how ready it is, which hard gates passed, who approved it, and what happens if production fails.